IoT Security Scanner: programma kody

Bellik: skanirleme diňe rugsat berlen lokal torlarda geçirilmelidir.

1. Programmanyň giriş nokady

Surat 1. Interfeýs skrinşoty üçin boş ýer

Baş sahypanyň skrinşoty. Bu ýerde programma açylanda görünýän esasy penjire, çep tarapdaky menýu we IoT Security Scanner ýazgysy görkezilmelidir.


```
# main.py
# Programmanyň esasy giriş nokady.
# Bu ýerde PyQt6 programmasy döredilýär we esasy penjire açylýar.

from PyQt6.QtWidgets import QApplication
import sys
from ui.main_window import MainWindow

if __name__ == "__main__":
    # QApplication grafiki interfeýsiň ähli wakalaryny dolandyrýar.
    app = QApplication(sys.argv)

    # Esasy penjire döredilýär.
    win = MainWindow()
    win.show()

    # Programma ulanyjy penjiräni ýapýança işleýär.
    sys.exit(app.exec())
```

2. Tor skanirlemesini ýerine ýetirýän modul

Surat 2. Interfeýs skrinşoty üçin boş ýer

Tor skanirlemesi sahypasynyň skrinşoty. Bu ýerde Wi-Fi/Ethernet adapteriniň saýlanyşy, IP aralygy, progress bar we tapylan gurluşlaryň tablisasy görkezilmelidir.


```
# core/scanner.py
# Bu modul IoT gurluşlaryny tapmak, açyk portlary barlamak
# we pluginleriň kömegi bilen howpsuzlyk töwekgelçiligini kesgitlemek üçin ulanylýar.

from PyQt6.QtCore import QThread, pyqtSignal
from concurrent.futures import ThreadPoolExecutor, as_completed
from pathlib import Path
import importlib.util
import ipaddress
import platform
import socket
import subprocess
import os

try:
    import nmap
except ImportError:
    nmap = None


class ScanThread(QThread):
    # UI bilen aragatnaşyk etmek üçin signallar.
    progress = pyqtSignal(int, str)
    device_found = pyqtSignal(dict)
    scan_finished = pyqtSignal(list)
    error = pyqtSignal(str)

    def __init__(self, ip_range="192.168.0.0/24", max_workers=10):
        super().__init__()
        self.ip_range = ip_range
        self.max_workers = max_workers
        self.devices = []
        self._stop_requested = False
        self.plugins = self.load_plugins()

    def run(self):
        # Skanirleme aýratyn akymda işleýär, şonuň üçin interfeýs doňmaýar.
        try:
            self.progress.emit(0, "Tor skanirlemesi başlandy...")
            hosts = self.discover_hosts()

            if not hosts:
                self.progress.emit(100, "Aktiw gurluş tapylmady.")
                self.scan_finished.emit([])
                return
```

```
            executor = ThreadPoolExecutor(max_workers=self.max_workers)
            futures = [executor.submit(self.scan_device, host) for host in hosts]
            completed = 0

            try:
                for future in as_completed(futures):
                    if self._stop_requested:
                        self.progress.emit(100, "Skanirleme saklandy.")
                        break

                    try:
                        device = future.result()
                        if device:
                            self.devices.append(device)
                            self.device_found.emit(device)
                    except Exception as exc:
                        self.error.emit(f"Skanirleme ýalňyşlygy: {exc}")

                    completed += 1
                    progress = int(completed / len(futures) * 100)
                    self.progress.emit(progress, f"Skanirlenýär... {progress}%")
            finally:
                if self._stop_requested:
                    for future in futures:
                        future.cancel()
                executor.shutdown(wait=True, cancel_futures=True)

            self.scan_finished.emit(self.devices)
        except Exception as exc:
            self.error.emit(f"Kritiki ýalňyşlyk: {exc}")

    def stop(self):
        # Ulanyjy "Stop" düwmesine basanda skanirleme saklanýar.
        self._stop_requested = True

    def discover_hosts(self):
        # Ilki bilen Nmap arkaly aktiw hostlar gözlenýär.
        try:
            if nmap is None:
                raise RuntimeError("python-nmap kitaphanasy gurulmady")

            nm = nmap.PortScanner(nmap_search_path=self.nmap_search_path())
            nm.scan(self.ip_range, arguments="-sn -n")
            hosts = list(nm.all_hosts())
            if hosts:
                return hosts
        except Exception as exc:
            self.progress.emit(
                0,
                f"Nmap elýeterli däl, göni IP barlagy ulanylýar: {exc}",
            )

        # Nmap işlemese, IP aralygy göni barlanýar.
        return self.hosts_from_range(self.ip_range)

    def nmap_search_path(self):
        # Windows ulgamynda Nmap-iň adaty gurnalýan ýerleri.
        paths = [
            Path("C:/Program Files (x86)/Nmap/nmap.exe"),
            Path("C:/Program Files/Nmap/nmap.exe"),
        ]
        existing = [str(path) for path in paths if path.exists()]
        return tuple(existing + ["nmap"])

    def hosts_from_range(self, ip_range):
        # CIDR görnüşindäki aralygy IP sanawyna öwürmek.
```

```
        try:
            network = ipaddress.ip_network(ip_range, strict=False)
            if network.num_addresses == 1:
                return [str(network.network_address)]
            return [str(ip) for ip in network.hosts()]
        except ValueError:
            return [ip_range.strip()] if ip_range.strip() else []

    def scan_device(self, host):
        # Bir gurluş barada esasy maglumatlar ýygnalýar.
        hostname = self.resolve_hostname(host)
        device = {
            "ip": host,
            "mac": self.get_mac_address(host),
            "hostname": hostname,
            "name": hostname if hostname != "Unknown" else host,
            "ports": [],
            "services": {},
            "vulnerable": False,
            "vulnerabilities": [],
            "recommendation": "",
            "recommendations": [],
            "risk_level": "low",
        }

        device["ports"] = self.scan_ports(host)
        device["services"] = self.identify_services(device["ports"])
        device = self.run_vulnerability_checks(device)
        return self.normalize_device(device)

    def scan_ports(self, host):
        # IoT gurluşlarynda köp duş gelýän portlar barlanýar.
        common_iot_ports = [
            21, 22, 23, 80, 443, 554, 8000, 8080,
            8888, 1883, 8883, 5683, 1900, 49152, 5353,
        ]
        open_ports = []

        for port in common_iot_ports:
            if self._stop_requested:
                break
            if self.is_port_open(host, port):
                open_ports.append(port)

        return open_ports

    def is_port_open(self, host, port, timeout=1.0):
        # TCP birikmesi arkaly portuň açykdygy barlanýar.
        try:
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
                sock.settimeout(timeout)
                return sock.connect_ex((host, port)) == 0
        except Exception:
            return False

    def identify_services(self, ports):
        # Port belgisine görä hyzmatyň ady kesgitlenýär.
        port_service_map = {
            21: "ftp",
            22: "ssh",
            23: "telnet",
            80: "http",
            443: "https",
            554: "rtsp",
            1883: "mqtt",
            8883: "mqtt/ssl",
            5683: "coap",
```

```
        }
        return {port: port_service_map.get(port, "unknown") for port in ports}

    def run_vulnerability_checks(self, device):
        # Her plugin gurluşy aýratyn barlaýar we netijäni täzeläp gaýtarýar.
        for plugin in self.plugins:
            try:
                device = plugin.check(device)
            except Exception as exc:
                self.error.emit(f"Plugin ýalňyşlygy {plugin.__name__}: {exc}")
        return device

    def normalize_device(self, device):
        # Netije UI, hasabat we maglumat bazasy üçin birmeňzeş formata getirilýär.
        vulnerabilities = device.get("vulnerabilities", [])
        recommendations = device.get("recommendations", [])
        device["vulnerable"] = bool(vulnerabilities) or bool(device.get("vulnerable"))

        if not device.get("recommendation"):
            if recommendations:
                device["recommendation"] = "; ".join(
                    rec.get("details") or rec.get("action") or str(rec)
                    for rec in recommendations
                )
            elif device["vulnerable"]:
                device["recommendation"] = (
                    "Gurluşyň sazlamalaryny barlaň we programma üpjünçiligini täzeläň."
                )
            else:
                device["recommendation"] = "Kritiki maslahat ýok."

        return device

    def load_plugins(self):
        # plugins bukjasyndaky ähli barlag modullary awtomatik ýüklenýär.
        plugins = []
        plugin_dir = Path(__file__).resolve().parent.parent / "plugins"
        plugin_dir.mkdir(parents=True, exist_ok=True)

        for file in os.listdir(plugin_dir):
            if file.endswith(".py") and file != "__init__.py":
                spec = importlib.util.spec_from_file_location(
                    f"plugins.{file[:-3]}",
                    plugin_dir / file,
                )
                module = importlib.util.module_from_spec(spec)
                spec.loader.exec_module(module)

                if hasattr(module, "check") and callable(module.check):
                    plugins.append(module)

        return plugins

    def get_mac_address(self, ip):
        # ARP tablisasyndan MAC salgyny almaga synanyşylýar.
        try:
            args = ["arp", "-a", ip] if platform.system() == "Windows" else ["arp", "-n", ip]
            result = subprocess.run(args, capture_output=True, text=True, timeout=3)
            for line in result.stdout.splitlines():
                if ip in line:
                    parts = line.split()
                    if len(parts) >= 3:
                        return parts[2]
        except Exception:
            pass
        return "Unknown"
```

```
    def resolve_hostname(self, ip):
        # IP salgysy boýunça gurluşyň ady kesgitlenýär.
        try:
            return socket.gethostbyaddr(ip)[0]
        except Exception:
            return "Unknown"
```

3. Netijeleri SQLite maglumat bazasynda saklamak

Surat 3. Interfeýs skrinşoty üçin boş ýer

Taryh ýa-da Panel sahypasynyň skrinşoty. Bu ýerde skanirleme netijeleriniň maglumatlar bazasynda saklanyp, soňra gaýtadan görkezilýändigi görkezilmelidir.


```
# database.py
# Bu modul skanirleme netijelerini SQLite maglumat bazasynda saklaýar.
# Şeýlelik bilen ulanyjy öňki barlaglaryň taryhyny görüp bilýär.

import sqlite3
from datetime import datetime
import os

DB_PATH = "iot_security.db"

SCAN_COLUMNS = {
    "timestamp": "TEXT",
    "ip": "TEXT",
    "name": "TEXT",
    "mac": "TEXT",
    "ports": "TEXT",
    "vulnerable": "BOOLEAN",
    "risk_level": "TEXT",
    "recommendation": "TEXT",
}


def init_db():
    # Maglumat bazasy we scans tablisasy döredilýär.
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute(
        '''
        CREATE TABLE IF NOT EXISTS scans (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp TEXT,
            ip TEXT,
            name TEXT,
            mac TEXT,
            ports TEXT,
            vulnerable BOOLEAN,
```

```
            risk_level TEXT,
            recommendation TEXT
        )
        '''
    )
    migrate_scans_table(cursor)
    conn.commit()
    conn.close()


def migrate_scans_table(cursor):
    # Köne bazalarda ýok sütünler bar bolsa, olar awtomatik goşulýar.
    cursor.execute("PRAGMA table_info(scans)")
    existing_columns = {row[1] for row in cursor.fetchall()}

    for column, column_type in SCAN_COLUMNS.items():
        if column not in existing_columns:
            cursor.execute(f"ALTER TABLE scans ADD COLUMN {column} {column_type}")


def save_scan(devices):
    # Skanirleme netijeleri maglumat bazasyna ýazylýar.
    init_db()
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    for device in devices:
        ports_str = ",".join(map(str, device.get("ports", [])))
        name = device.get("name") or device.get("hostname") or device.get("ip", "Unknown")

        cursor.execute(
            '''
            INSERT INTO scans
            (timestamp, ip, name, mac, ports, vulnerable, risk_level, recommendation)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
            ''',
            (
                timestamp,
                device.get("ip", "Unknown"),
                name,
                device.get("mac", "Unknown"),
                ports_str,
                device.get("vulnerable", False),
                device.get("risk_level", "low"),
                device.get("recommendation", ""),
            ),
        )

    conn.commit()
    conn.close()


def load_history():
    # Öňki skanirlemeleriň taryhy okalýar.
    init_db()
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    cursor.execute("SELECT DISTINCT timestamp FROM scans ORDER BY timestamp DESC")
    timestamps = [row[0] for row in cursor.fetchall()]

    history = []
    for timestamp in timestamps:
        cursor.execute(
            '''
            SELECT ip, name, mac, ports, vulnerable, risk_level, recommendation
```

```
            FROM scans WHERE timestamp = ? ORDER BY ip
            ''',
            (timestamp,),
        )

        devices = []
        for row in cursor.fetchall():
            ports = [int(port) for port in row[3].split(",")] if row[3] else []
            devices.append(
                {
                    "ip": row[0],
                    "name": row[1],
                    "mac": row[2],
                    "ports": ports,
                    "vulnerable": bool(row[4]),
                    "risk_level": row[5],
                    "recommendation": row[6],
                }
            )

        history.append({"timestamp": timestamp, "devices": devices})

    conn.close()
    return history


def get_scan_stats():
    # Baş sahypada görkezilýän umumy statistika taýýarlanýar.
    init_db()
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    cursor.execute("SELECT COUNT(DISTINCT timestamp) FROM scans")
    total_scans = cursor.fetchone()[0] or 0

    cursor.execute("SELECT COUNT(*) FROM scans")
    total_devices = cursor.fetchone()[0] or 0

    cursor.execute("SELECT COUNT(*) FROM scans WHERE vulnerable = 1")
    vulnerable_devices = cursor.fetchone()[0] or 0

    cursor.execute("SELECT MAX(timestamp) FROM scans")
    last_scan = cursor.fetchone()[0]

    conn.close()
    return {
        "total_scans": total_scans,
        "total_devices": total_devices,
        "vulnerable_devices": vulnerable_devices,
        "last_scan": last_scan,
    }
```

4. Telnet gowşak parollaryny barlaýan plugin

Surat 4. Interfeýs skrinşoty üçin boş ýer

Telnet gowşaklygy ýüze çykarylan ýagdaýyň skrinşoty. Bu ýerde 23-nji port, ýokary töwekgelçilik derejesi we paroly üýtgetmek boýunça maslahat görkezilmelidir.


```
# plugins/telnet_weak_auth.py
# Bu plugin diňe rugsat berlen lokal torlarda ulanmak üçin niýetlenendir.
# Ol Telnet hyzmatynda köp duş gelýän standart login/parol jübütlerini barlaýar.

import telnetlib
import logging

logger = logging.getLogger(__name__)


def check(device):
    # Telnet porty açyk bolmasa, barlag geçirilmeýär.
    if 23 not in device.get("ports", []):
        return device

    weak_credentials = [
        ("admin", "admin"),
        ("root", "root"),
        ("user", "user"),
        ("admin", "password"),
        ("root", "123456"),
        ("admin", "1234"),
    ]

    for username, password in weak_credentials:
        if test_telnet_login(device["ip"], username, password):
            vulnerability = {
                "type": "telnet_weak_auth",
                "severity": "high",
                "description": f"Telnet gowşak paroly: {username}/{password}",
                "fix_available": True,
                "fix_method": "change_password",
            }

            recommendation = {
                "action": "Telnet parolyny üýtgetmek",
                "priority": "high",
                "details": (
                    f"Gowşak parol tapyldy: {username}/{password}. "
                    "Telnet-i öçüriň ýa-da güýçli parol ulanyň."
                ),
            }

            device.setdefault("vulnerabilities", []).append(vulnerability)
            device.setdefault("recommendations", []).append(recommendation)
```

```
            device["vulnerable"] = True
            device["risk_level"] = update_risk_level(device.get("risk_level", "low"), "high")

    return device


def test_telnet_login(ip, username, password, timeout=5):
    # Telnet arkaly giriş synagy geçirilýär.
    try:
        tn = telnetlib.Telnet(ip, 23, timeout=timeout)

        index, _, _ = tn.expect([b"login: ", b"username: ", b"Login: "], timeout=timeout)
        if index >= 0:
            tn.write(username.encode("ascii") + b"\n")

        index, _, _ = tn.expect([b"password: ", b"Password: "], timeout=timeout)
        if index >= 0:
            tn.write(password.encode("ascii") + b"\n")

        index, _, _ = tn.expect([b"Welcome", b"#", b"$", b">"], timeout=timeout)
        tn.close()
        return index >= 0
    except Exception as exc:
        logger.debug(f"Telnet giriş synagy başartmady: {ip}: {exc}")
        return False


def update_risk_level(current_level, new_level):
    # Has ýokary töwekgelçilik derejesi saýlanýar.
    levels = {"low": 0, "medium": 1, "high": 2, "critical": 3}
    return max([current_level, new_level], key=lambda level: levels.get(level, 0))
```

5. Web-interfeýsli IoT gurluşlaryny barlamak

Surat 5. Interfeýs skrinşoty üçin boş ýer

Web-interfeýsli IoT gurluşynyň skrinşoty. Bu ýerde HTTP/HTTPS portly kamera ýa-da IoT kandidat gurluşy we firmware/standart hasaplar boýunça maslahat görkezilmelidir.


```
# plugins/hikvision.py
# Bu plugin HTTP porty açyk bolan kameralary ýa-da web-paneli bolan IoT gurluşlaryny belleýär.
# Netijede programma ulanyja firmware we giriş maglumatlaryny barlamagy maslahat berýär.


def check(device):
    if 80 in device.get("ports", []):
        vulnerability = {
            "type": "possible_iot_http_interface",
            "severity": "medium",
            "description": (
```

```
                "HTTP web-interfeýsi açyk. Firmware we standart login/parol barlanmaly."
            ),
            "fix_available": False,
            "fix_method": "manual_review",
        }

        recommendation = {
            "action": "Web-interfeýsi barlamak",
            "priority": "medium",
            "details": (
                "Firmware-i täzeläň, standart ulanyjy hasaplaryny öçüriň "
                "we HTTP hyzmatyny daşarky tora açmaň."
            ),
        }

        device.setdefault("vulnerabilities", []).append(vulnerability)
        device.setdefault("recommendations", []).append(recommendation)
        device["risk_level"] = "medium"

    return device
```

6. PDF hasabatyny döretmek

Surat 6. Interfeýs skrinşoty üçin boş ýer

PDF hasabat döretmek prosesiniň skrinşoty. Bu ýerde hasabaty eksport etmek düwmesi, faýl saýlamak penjiresi ýa-da döredilen PDF hasabaty görkezilmelidir.


```
# reports.py
# Bu modul skanirleme netijeleri boýunça PDF görnüşinde hasabat döredýär.

from pathlib import Path
import tempfile

from matplotlib.figure import Figure
from reportlab.lib.pagesizes import A4
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.pdfgen import canvas


def get_report_font():
    # Kiril we milli harplar üçin Windows şrifti ulanylýar.
    font_paths = [
        Path("C:/Windows/Fonts/arial.ttf"),
        Path("C:/Windows/Fonts/segoeui.ttf"),
    ]

    for font_path in font_paths:
        if font_path.exists():
```

```
            pdfmetrics.registerFont(TTFont("IoTReportFont", str(font_path)))
            return "IoTReportFont"

    return "Helvetica"


def generate_pdf(devices, filename="report.pdf"):
    c = canvas.Canvas(filename, pagesize=A4)
    font_name = get_report_font()
    c.setFont(font_name, 12)
    y = 800

    c.drawString(50, y, "IoT Security Report")
    y -= 30

    for device in devices:
        # Sahypa dolsa, täze sahypa açylýar.
        if y < 90:
            c.showPage()
            c.setFont(font_name, 12)
            y = 800

        name = device.get("name") or device.get("hostname") or "Unknown"
        ip = device.get("ip", "Unknown")
        ports = ", ".join(map(str, device.get("ports", []))) or "ýok"
        vulnerable = bool(device.get("vulnerable") or device.get("vulnerabilities"))
        recommendation = device.get("recommendation") or "Kritiki maslahat ýok."

        c.drawString(50, y, f"Gurluş: {name} ({ip})")
        y -= 20
        c.drawString(50, y, f"Portlar: {ports}")
        y -= 20
        c.drawString(50, y, f"Gowşaklyk: {'Hawa' if vulnerable else 'Ýok'}")
        y -= 20
        c.drawString(50, y, f"Maslahat: {recommendation[:110]}")
        y -= 30

    if devices:
        # Howpsuz we gowşak gurluşlaryň paýy diagramma görnüşinde görkezilýär.
        fig = Figure(figsize=(3, 2))
        ax = fig.add_subplot(111)
        safe = sum(1 for d in devices if not (d.get("vulnerable") or d.get("vulnerabilities")))
        vuln = len(devices) - safe
        ax.pie(
            [safe, vuln],
            labels=["Safe", "Vulnerable"],
            colors=["#66FF66", "#FF6666"],
            autopct="%1.1f%%",
        )

        with tempfile.NamedTemporaryFile(suffix=".png", delete=False) as tmp:
            chart_path = tmp.name

        try:
            fig.savefig(chart_path, bbox_inches="tight")
            c.drawImage(chart_path, 50, y - 200, width=200, height=150)
        finally:
            Path(chart_path).unlink(missing_ok=True)

    c.save()
```

7. Interfeýsde skanirlemäni başlatmak

Surat 7. Interfeýs skrinşoty üçin boş ýer

Skanirlemäni başlatmak sahypasynyň skrinşoty. Bu ýerde skanirleme sazlamalary, başlatmak/saklamak düwmeleri, log meýdany we netijeleriň real wagtda goşulmagy görkezilmelidir.


```
# ui/pages.py bölegi
# Bu bölekde "Başla" düwmesi skanirleme akymyny döredýär,
# netijeleri tablisa ýazýar we skanirleme gutaranda maglumat bazasyna saklaýar.

def start_scan():
    ip_range = ip_range_input.text().strip()

    if not validate_ip_range(ip_range):
        QMessageBox.warning(
            widget,
            "Nädogry IP aralygy",
            "IP ýa-da tor aralygyny 192.168.0.0/24 görnüşinde giriziň.",
        )
        return

    widget.last_devices = []
    results_table.setRowCount(0)
    log_output.clear()
    progress_bar.setVisible(True)
    progress_bar.setValue(0)
    start_btn.setEnabled(False)
    stop_btn.setEnabled(True)

    append_log(f"Skanirleme başlandy: {ip_range}")

    thread = ScanThread(ip_range=ip_range, max_workers=workers_for_intensity())
    thread.progress.connect(on_progress)
    thread.device_found.connect(add_device_to_table)
    thread.error.connect(on_error)
    thread.scan_finished.connect(on_finished)

    widget.scan_thread = thread
    thread.start()


def add_device_to_table(device):
    # Tapylan gurluşyň maglumatlary tablisa goşulýar.
    row = results_table.rowCount()
    results_table.insertRow(row)

    ports = ", ".join(map(str, device.get("ports", []))) or "ýok"
    vulnerable = "Hawa" if device.get("vulnerable") else "Ýok"

    values = [
        device.get("ip", "Unknown"),
```

```
        device.get("name") or device.get("hostname", "Unknown"),
        device.get("mac", "Unknown"),
        ports,
        device.get("risk_level", "low"),
        vulnerable,
        device.get("recommendation") or "Kritiki maslahat ýok.",
    ]

    for column, value in enumerate(values):
        results_table.setItem(row, column, QTableWidgetItem(str(value)))


def on_finished(devices):
    # Skanirleme gutarandan soň netije saklanýar.
    widget.last_devices = devices
    start_btn.setEnabled(True)
    stop_btn.setEnabled(False)
    progress_bar.setValue(100)

    if devices:
        save_scan(devices)
        append_log(f"Skanirleme tamamlandy. Saklanan gurluşlar: {len(devices)}")
    else:
        append_log("Skanirleme tamamlandy, aktiw gurluş tapylmady.")
```